Sub-second Compressive-Sensing Image Decryption on a Consumer GPU

Profiling-driven, bit-exact optimization of a batched PnP-ADMM super-resolution decoder

Oct 26, 2026 · @lordan

Abstract

We cut the end-to-end decryption time of a 12-megapixel compressive-sensing (CS) image from **6.1 s to 2.13 s with a byte-identical output**, and to **0.95 s** with a decode preset that costs 0.12 dB PSNR and 0.0002 SSIM. The decoder reconstructs 3,969 overlapping tiles by reweighted-L1 FISTA, then super-resolves each tile 2× with a plug-and-play ADMM loop whose proximal step is OpenCV-exact non-local means (NLM), all batched on one AMD Radeon RX 7900 XT. Profiling showed that the GPU kernels were less than half of the original time: a quadratic host-side anchor lookup (1.6 s), serial CPU passes that did nothing (0.35 s), PNG encoding (0.27 s) and driver allocation all sat on the critical path. We describe 20 optimizations, each verified bit-for-bit against its predecessor, including an integer-exact NLM that uses packed-byte dot products (4.1× faster), a wavefront formulation of error-diffusion dithering, a stack-ordered device arena reserved during key derivation, and a parallel PNG writer. We also report the optimizations that did not pay off, a measured but unexplained 8% gain from marker events on ROCm/Windows, and a quality sweep showing that tile overlap and inner iterations, unlike ADMM rounds, can be reduced almost for free. All timings exclude the optional neural (Swin2SR) prior, which this workload does not run.

1. Introduction

A CS image container stores a key-selected subset of pixel samples (here 90% of a 2016 × 1512 luma grid plus quarter-resolution chroma, encrypted and authenticated); decryption regenerates the sample positions from a password-derived key and solves a sparse inverse problem to rebuild a 4032 × 3024 image. Reconstruction quality comes from expensive iterative solvers, so decode latency, not encode latency, is what users wait for. Before this work, the batched GPU decoder of the ycc420 container with 300,000 exact high-resolution anchors took 6.1 s; the goal was under one second end to end, process start to PNG on disk.

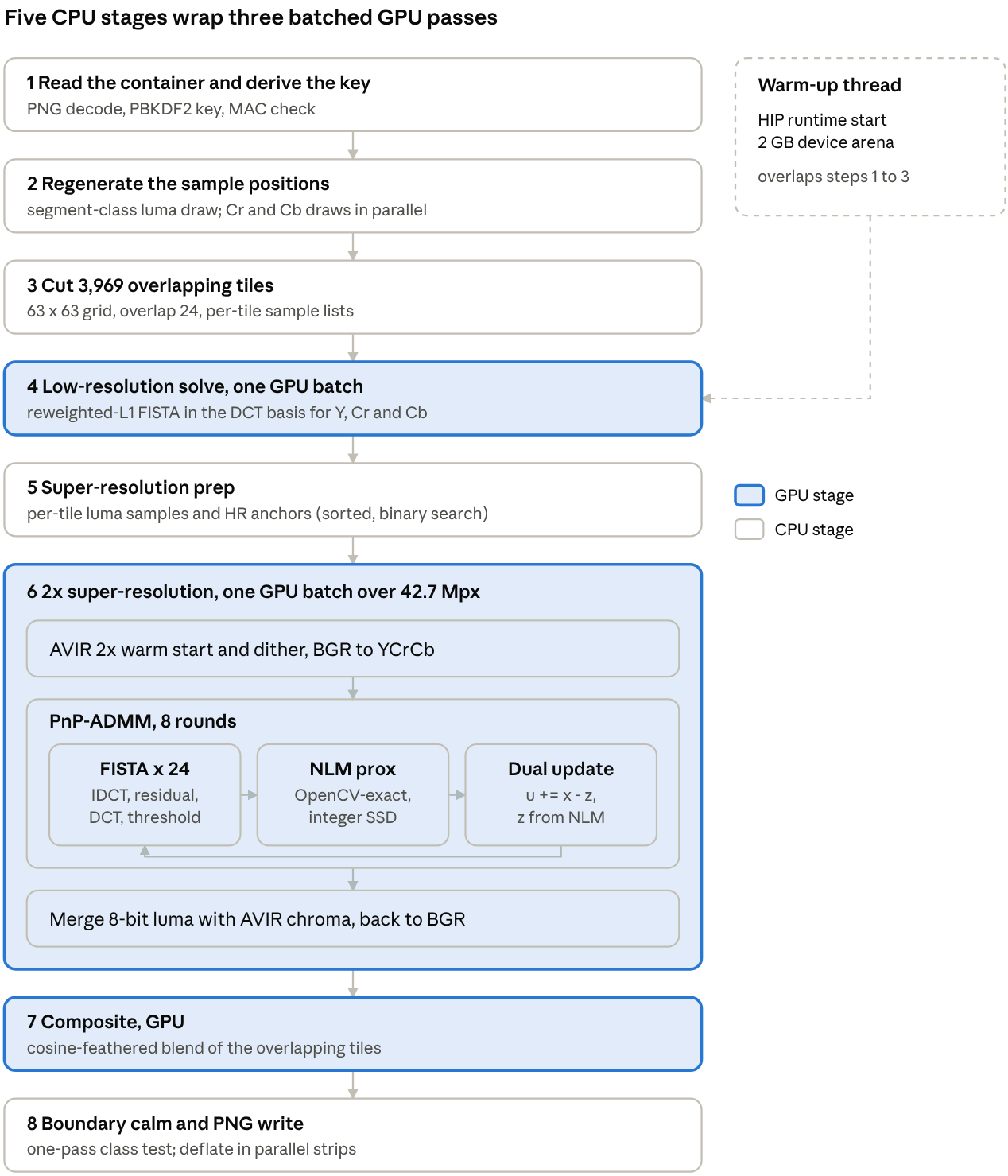
The work was driven by measurement rather than by assumptions about where time goes. The main finding is that a GPU decoder of this kind is not GPU-bound until its host side is cleaned up: half of the original 6.1 s was CPU work, much of it either quadratic or useless.

Contributions:

1. A stage-level profile of a batched CS + PnP-ADMM super-resolution decoder on a consumer GPU, and the methodology used to get it without hardware counters.
2. Twenty optimizations, every one verified to leave the decoded image byte-identical, that take the decode from 6.1 s to 2.13 s (2.9×).
3. An integer-exact GPU formulation of OpenCV's `fastNlMeansDenoising` using a sum-of-squares identity and packed-byte dot products, 4.1× faster than the direct kernel.
4. A bit-exact wavefront formulation of AVIR's serial error-diffusion dither.
5. A quality–speed sweep of the decode knobs, and a `--fast` preset that reaches 0.95 s at a 0.12 dB cost.
6. A record of the optimizations that did not help, and why.

2. System overview

The decoder runs eight stages; three of them are single batched GPU passes over all 3,969 tiles, the other five run on the CPU.



decode pipeline of the ycc420 container, default settings

The GPU passes are batched because each tile is a small problem (about 48 × 56 pixels before, 96 × 112 after super-resolution): one launch covers every tile, with the tile index in `blockIdx.y`. Neighbour warm starts were dropped earlier so that tiles are independent.

Low-resolution solve. Each tile plane x is recovered from its samples b by reweighted-L1 FISTA with an orthonormal DCT basis Ψ, written as two dense matrix products per transform (cheaper than an FFT at about 100 px).

Super-resolution. The 2× luma is refined by plug-and-play ADMM. In equation (1), A takes the 2 × 2 average of the HR tile at the LR sample positions, whose values are b; the second term pins the anchored HR pixels (set H, values a) with the anchor weight `--hr-weight`; W holds the per-frequency weights, Ψ is the DCT, λ the sparsity weight and ρ the ADMM penalty. Each round runs 24 FISTA steps on (1), then the NLM denoiser and the dual update (2):

$$x^{k+1} = \arg\min_x \|Ax - b\|_2^2 + w_h \|x_H - a_H\|_2^2 + \lambda \|W\Psi x\|_1 + \rho \|x - (z^k - u^k)\|_2^2 \quad (1)$$

$$z^{k+1} = D_h(x^{k+1} + u^k), \qquad u^{k+1} = u^k + x^{k+1} - z^{k+1} \quad (2)$$

PnP-ADMM round k: data, anchor, sparsity and coupling terms (1); NLM prox and dual update (2)

The denoiser D in (2) is OpenCV's `fastNlMeansDenoising` with strength h, a 7 × 7 template and a 15 × 15 search window, applied to the 8-bit quantization of x + u, so the GPU and CPU paths agree to the last bit. The default budget is 8 rounds of 24 steps.

3. Experimental setup and methodology

Every number in this paper comes from one machine, one image and one command, timed as wall clock from process start to exit.

Item	Value
CPU	AMD Ryzen 9 7900, 12 cores / 24 threads
GPU	AMD Radeon RX 7900 XT (gfx1100, 84 CUs, 20 GB)
Software	Windows 11, ROCm/HIP 7.2, clang-cl (LLVM 23), OpenCV 4.9, AOCL-Compression zlib
Image	IMG_3690, 4032 × 3024 output (12.2 MP)

Item	Value
Container	seg8-tile, sampling ratio 0.9, ycc420, 64 segment classes
Anchors	300,000 exact HR luma samples (sidecar, used inside the solve)
Command	`decrypt --no-preview --tv 0 --photo-upscaler cs-sr --sr-pnp admm --hr-weight 1`
Auto grid	63 × 63 = 3,969 tiles, overlap 24, LR tiles about 48 × 56, HR tiles about 96 × 112
Solver budget	LR: reweighted FISTA; SR: 8 ADMM rounds × 24 FISTA steps on 42.7 Mpx of overlapped HR tiles

Timing. Wall time was measured by the calling shell around the whole process, unprofiled, as the median of 5 to 7 runs after one warm-up run. Run-to-run spread was about ±20 ms; differences under 30 ms were confirmed by running both variants back to back.

Exactness criterion. A change was accepted as exact only if the decoded PNG matched the previous output byte for byte at pixel level (maximum absolute difference 0 over all 36.6 M channel values). One change that failed this test on the first attempt (Section 5.6) was fixed rather than accepted. The `--fast` preset is the only intentional output change, and is evaluated separately (Section 7).

Quality metrics. PSNR against the original, SSIM (structure), CIEDE2000 ΔE at the 95th percentile (colour), following the project's rule of judging sharpness by SSIM and colour by ΔE, with PSNR as a tiebreaker.

Profiling without hardware counters. ROCm on Windows ships no `rocprofv3`, `roctracer` or `rocm-smi`, so three instruments were used:

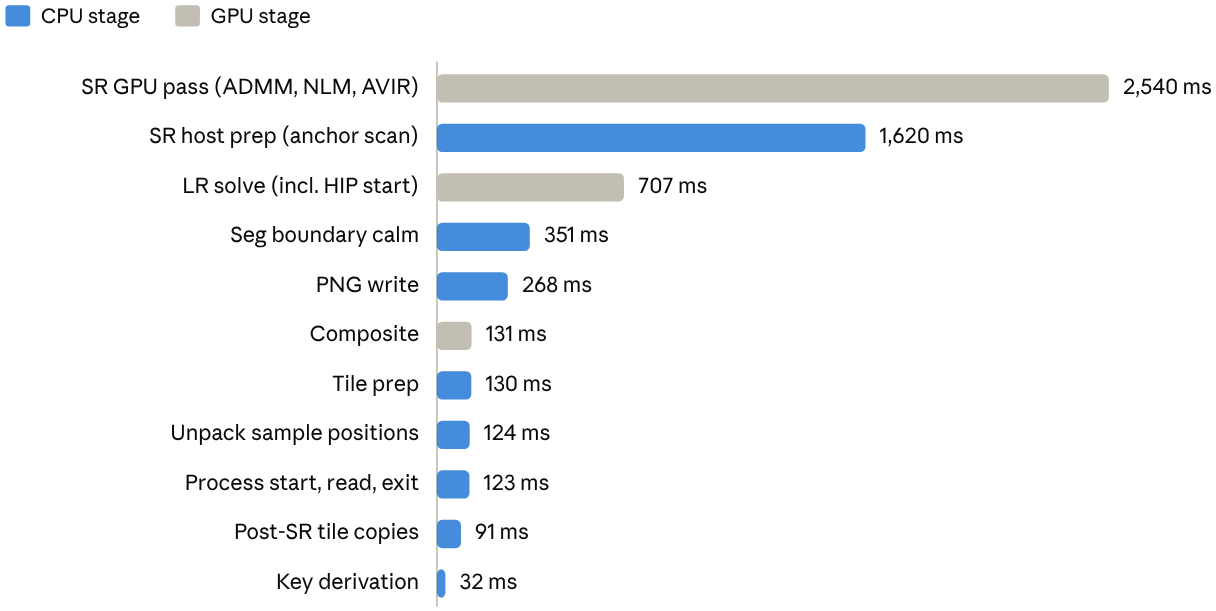
- *Stage marks*: temporary timestamps at about 30 points of the pipeline, each preceded by `hipDeviceSynchronize()` in GPU code so the mark includes the queued work. They were removed before the final measurements.
- *Per-kernel events*: the existing `CS_GPU_PROF=1` mode brackets every ADMM launch with a `hipEvent` pair. Events add overhead, so they were used for proportions; absolute numbers come from unprofiled runs.
- *Compiler resource reports*: `-Rpass-analysis=kernel-resource-usage` for VGPR count, scratch spills and occupancy of each kernel.

4. Baseline characterization

The baseline decode took 6.1 s, and the profile contradicted the assumption that it was GPU-bound: CPU stages took 2.7 s, and the second-largest stage was a host loop.

2.7 s of the 6.1 s decode was CPU work, led by a 1.6 s anchor scan

Baseline stage times, 6,117 ms in total: 2,739 ms in CPU stages, 3,378 ms in GPU stages



stage marks with device syncs, baseline build, one profiled run (6.12 s wall)

The SR GPU pass itself was 2.54 s, of which 2.27 s were ADMM kernels run as two serial batches (tiles with and without anchors). Per-kernel events split those 2.27 s as follows: NLM 1,128 ms (50%), the forward-pass-B GEMM with the FISTA step epilogue 460 ms (20%), and the three other DCT GEMMs 184 to 222 ms each. The NLM kernel ran 70 ms per launch, 8 launches per batch. The remaining 270 ms of the pass were host-side layout, device allocations and the AVIR warm start.

Three stages stood out as fixable without touching the numerics: the anchor scan (1.62 s), the seg boundary calm (351 ms, which on this image changed nothing) and the PNG write (268 ms, single-threaded). Together they were 37% of the decode.

5. Bit-exact optimizations

Twenty changes took the default decode from 6.1 s to 2.13 s without changing a single output byte. They fall into four groups: host algorithms on the critical path, GPU kernels, GPU orchestration, and CPU stages and I/O. For each we give the mechanism, why the

output cannot change, and the measured gain.

5.1 Host algorithms on the critical path

Anchor lookup (1.62 s → 0.03 s). Before handing each tile to the GPU, the host collected the anchors that fall inside it by scanning the whole 300,000-anchor list, once per tile: 3,969 × 300,000 = 1.2 billion integer divisions. The anchors are now stable-sorted once by global index; a tile visits its rows and finds each row's anchors with two binary searches (about 380,000 searches in total). *Exact:* the same anchors are found, and a stable sort keeps duplicates in list order, so the last one still wins when they are scattered into the tile map. The per-tile preparation loop itself also became an OpenMP loop.

Seg boundary calm (351 ms → 1.5 ms). After compositing, a pass softens boundaries of low-density segment classes. To decide which classes are low-density it ran, for each of 38 present classes, a full-frame comparison, two `countNonZero` calls and a `bitwise_and`: about 150 passes over a 12 MP frame. On this image no class qualified, so the 351 ms changed nothing. One parallel pass now counts area and sampled pixels per class; the expensive path runs only if a class is below 40%. *Exact:* the decision uses the same counts and the same double-precision ratio. Counting happens on the label grid when the image is exactly 1× or 2× of it, because nearest-neighbour 2× replicates each label 2 × 2, so both counts scale by 4 and every ratio is unchanged.

Segment-mask index draw (127 ms → 64 ms). Regenerating the luma sample positions requires per-class boundary bands (a distance transform per class), an adjacent-class boost (a dilation per class) and per-class pixel pools, all computed serially before a sequential Mersenne-Twister draw. Bands and boosts are independent per class and now run in parallel; all pools are filled in one raster pass; the two final keyed shuffles of the row and column lists run on two threads. The two chroma index draws, independent of the luma draw, run concurrently with it. *Exact:* the random stream is consumed in the same order, and each pool keeps raster order.

5.2 GPU kernels

One ADMM batch instead of two (≈300 ms). Tiles with and without anchors were solved as two separate GPU batches, because the FISTA step length depends on the anchor weight. The fixed-step epilogue now picks the step per tile (`hrt[tile] ? step_hr : step`). *Exact:* each tile gets the step its own batch would have used; tiles never interact.

Non-local means prox (1,128 ms → 273 ms). The ADMM proximal step is OpenCV's `fastNlMeansDenoising(h, 7, 15)` on 8-bit luma, reproduced bit for bit (integer SSD, `>> 6` distance quantization, fixed-point weights). The original kernel computed each pixel's 225 template SSDs directly: 49 squared differences and 98 LDS byte loads per offset. Three exact reformulations followed:

1. *Sliding row sums.* A thread owns 16 output rows of one column. For each offset it computes 22 seven-wide row SSDs and slides a seven-row window down them, so each row sum is reused by up to 7 outputs.
2. *Weight table.* `w = round(37426 · exp(−d/h²))` depends only on `idx = ssd >> 6`. Each block tabulates it once; past index `6.95 h² / 1.306` every weight is below the 38 cut-off and is zero. The host sizes the table from the batch's largest h.
3. *Packed-byte cross term.* Each row sum uses the identity below. The template row's 7 bytes and its sum of squares stay in registers for all 225 offsets; the shifted row is three LDS dwords realigned with `v_alignbyte`, and the cross term and the shifted row's squares are four `v_dot4_u32_u8`. The centre pixel is byte 3 of a shifted row fetched three steps earlier, kept in a register ring.

$$\Sigma\,(a_t - b_t)^2 = \Sigma\,a_t^2 + \Sigma\,b_t^2 - 2\,\Sigma\,a_t\,b_t \tag{3}$$

each sum runs over the 7 template columns, t = 0 … 6

integer identity behind the packed-byte NLM: template row a, shifted row b

Exact: all arithmetic is integer; the identity holds exactly; the weight table holds the same `__float2int_rn(expf(…))` values the direct kernel computes. Section 6 shows that the second step, on its own, made the kernel twice as slow.

LR tile solve on the tiled GEMM (692 ms → 497 ms). The low-resolution FISTA stage still used one-thread-per-output DCT kernels, while the ADMM stage had long used an LDS-tiled GEMM. Switching them was free: both accumulate `acc += a·b` in ascending k with fused multiply-add from zero, so each sum is the same sequence of FMAs.

Leaner fixed-step ADMM (−140 ms). Three of the 14 full-size device buffers were redundant on the fixed-step path: `dX` was only a staging copy, `dZC` always equalled `dXP`, and the frequency weights `dWS` are a closed-form ramp. Computing the weight in the epilogue with the same expression as the `k_ws` kernel, and dropping the duplicate write, cut the step epilogue from 2.4 to 1.8 ms per launch and freed 0.5 GB.

Wavefront error diffusion (35 ms → 3 ms). AVIR's dither diffuses quantization error right and to the three pixels below, so the original kernel ran one thread per tile and channel, serially over about 10,000 pixels. Row i only needs row i−1 two columns ahead, so one thread per row now runs column s − 2i on step s, with one carry row per image row in LDS. *Exact only after a fix:* see Section 5.6.

5.3 GPU orchestration

HIP warm-up and device arena (≈230 ms). HIP runtime start-up (about 130 ms) and `hipMalloc` of about 3 GB of buffers (about 100 ms) sat on the critical path. A thread started at decode entry now initializes HIP and reserves one 2 GB arena while the main thread reads the PNG, derives the key and unpacks the samples. `DevBuf` carves 256-byte-aligned pieces from the top of the arena, and a free marks its piece dead and lets the top retreat over dead pieces. Local buffers die in reverse order of creation, so the arena behaves as a stack; overflow falls back to `hipMalloc`. The 2 GB size was chosen because a 4 GB reservation finished after the first GPU stage needed it (Section 6).

Marker events (454 → 419 ms ADMM span). Unprofiled decodes were consistently 30 ms slower than profiled ones. Bracketing every ADMM launch with a timing-free event reproduces the profiled timing; recording one only after each launch does not. We did not find the mechanism (no driver-level tracing is available on this platform), so the code comment calls it measured and unexplained.

Fewer copies (≈110 ms). The SR output tiles are now views into one host buffer, so they download in a single copy, the compositor uploads them without repacking, and the 3,969 frees become one. The compositor uploads each distinct feather-weight map once (a handful) instead of once per tile (80 MB). Host packing loops in the GPU modules run on all cores via a small `parallel_for`.

5.4 CPU stages and I/O

Parallel PNG writer (272 ms → 24 ms). OpenCV already writes PNG with its fastest settings (SUB filter, deflate level 1, RLE strategy), on one thread. The new writer applies the same filter and the same zlib settings to 64-row strips on all cores. Each strip is raw

deflate ended by a sync flush (byte-aligned, not final), the last one by a finish; the strips go into consecutive IDAT chunks, with the zlib header on the first and the Adler-32, combined with `adler32_combine`, after the last. The file size is unchanged (12.4 MB) and the decoded pixels are identical. It links AMD's AOCL-Compression zlib, so it is compiled only in the GPU build.

Tile preparation and teardown. Building the 3,969 per-tile containers is now an OpenMP loop (130 → 56 ms), with the order-dependent segment filter kept in a serial pre-pass. The per-tile containers, tens of thousands of small allocations, are freed on a background thread that overlaps the calm pass and the PNG write.

5.5 Summary of the exact changes

Change	Stage	Before	After
Sorted anchors + binary search	SR host prep	1,620 ms	30 ms
NLM: row sums, table, dot4	ADMM kernel	1,128 ms	273 ms
Seg calm one-pass test	CPU post	351 ms	1.5 ms
One ADMM batch	ADMM	2 serial passes	1 pass, −300 ms
PNG writer	I/O	272 ms	24 ms
LR solve on tiled GEMM	LR stage	692 ms	497 ms
HIP warm-up + arena	setup	≈230 ms on path	hidden
Seg mask draw	unpack	127 ms	64 ms
Lean ADMM buffers	ADMM	2.4 ms/step	1.8 ms/step
Copies, pooling, packing	SR + composite	≈150 ms	≈40 ms
Wavefront dither	SR	35 ms	3 ms
Marker events	ADMM	454 ms	419 ms (fast grid)

5.6 The one change that failed the exactness test

The first wavefront dither differed from the serial kernel in 0.002% of values (at most 3 levels). In the serial kernel, the carry to the right neighbour `L[j+1] += noise · 0.364842` sits in the same loop iteration as its multiply, so the compiler fuses it into one

FMA; in the wavefront version the carry crossed iterations and was rounded separately. Carrying the raw noise and applying `__fmaf_rn` at the use restored bit-exactness. The lesson: "same expression" is not "same rounding" under default floating-point contraction, and the shared epilogue math of the GEMM kernels was moved into common device functions for the same reason.

6. Negative and inconclusive results

Five attempts did not deliver what their reasoning predicted; each taught something about where the real limit was.

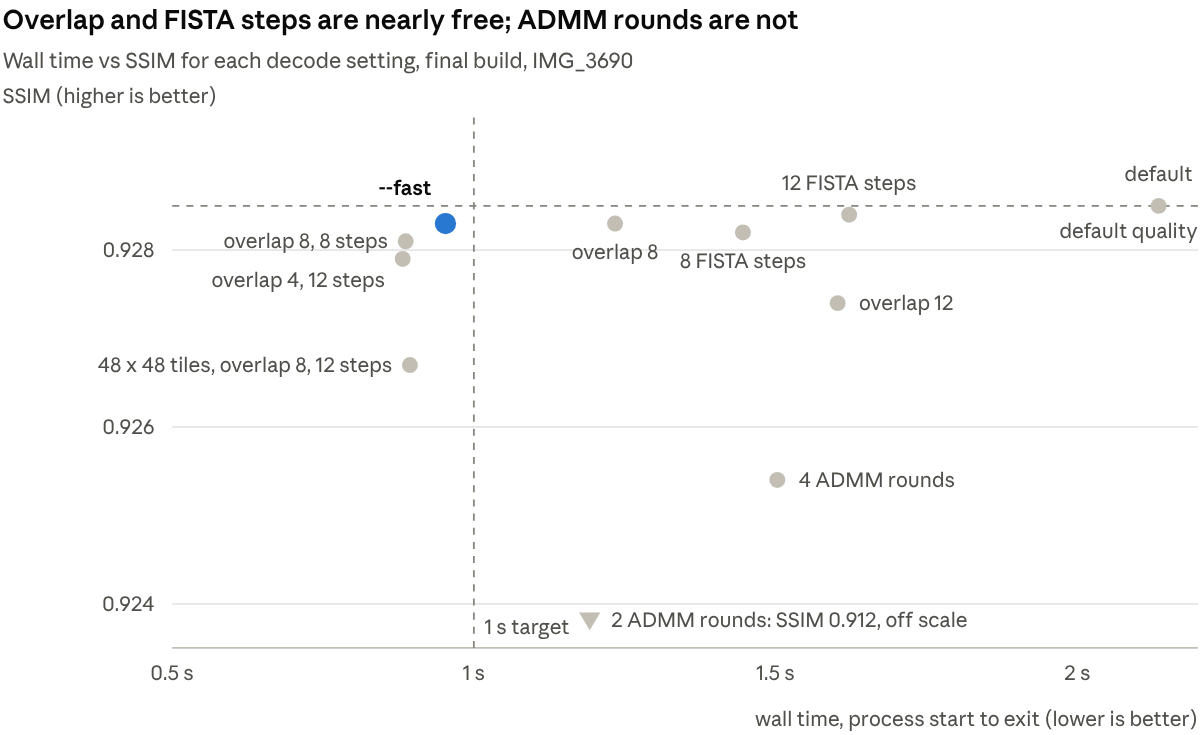
Attempt	Prediction	Measured	Why
NLM with sliding row sums, fully unrolled	4–5× faster	2× slower (1.13 → 2.33 s)	the compiler hoisted all 154 unrolled LDS loads: 256 VGPRs, 284 B/lane scratch spills, occupancy 5. A shifting 7-entry ring plus `sched_barrier` per output row brought it to 105 VGPRs and 0.48 s
NLM with an LDS table of window sums of squares	fewer LDS loads per offset	no change (0.49 → 0.52 s)	the kernel was bound by LDS bandwidth and occupancy, not by load count: the 19 KB table cut blocks per WGP. Moving the sums of squares into registers (dot4 of the bytes with themselves) gave 0.27 s
Sparser scheduling barriers in NLM	more overlap of LDS latency	no change	same cause as above

Attempt	Prediction	Measured	Why
Fused GEMM pairs (`k_gemm2` , 32-row LDS stripe)	fewer launches and half the intermediate traffic	943 vs 965 ms, within noise	the DCT GEMMs run at about 9 TFLOP/s, limited by the FMA inner loop; the fused kernel needs 178–185 VGPRs (occupancy 4). Kept: it is exact and halves launches
A single flush event after each launch	reproduce the profiled speed-up	no gain	only a marker before and after each launch helps (Section 5.3)
4 GB device arena	hide all allocations	first GPU stage waited 30 ms	a 4 GB reservation took longer than the CPU-only window; 2 GB fits it

Two broader observations. First, the GEMM kernels are the remaining wall for an exact decode: neither fusion, nor fewer launches, nor less memory traffic moved them. Halving their arithmetic needs the even/odd factorization of the DCT matrix, which changes rounding and therefore the output. Second, two of the five failures came from reasoning about instruction counts on a GPU whose limit was register pressure and LDS occupancy; the compiler resource report was the fastest diagnostic.

7. Quality–speed trade-off

At the default settings the exact optimizations stop at 2.13 s, so reaching one second requires less work, and the question is which knob costs the least quality. Two knobs are almost free and one is not.



final build, median of 3 runs per setting; SSIM against IMG_3690.png

The three knobs act on different parts of the work:

- **Tile overlap** (default 24 px on the LR grid) sets the redundancy: at overlap 24 the 3,969 HR tiles cover 42.7 Mpx for a 12.2 Mpx image (3.5×); at overlap 8 they cover 20.3 Mpx. Every GPU stage scales with it. The feathered blend still has 16 HR pixels to hide seams.
- **Inner FISTA steps** (default 24, from `--iterations 6`) are the steps per ADMM round. The data term converges within about 12; the extra steps barely move the result.
- **ADMM rounds** (default 8) are the denoise–data alternations. Each round is where the NLM prior acts, and removing rounds loses structure: SSIM falls to 0.9254 at 4 rounds and 0.9123 at 2.

Setting	PSNR (dB)	ΔE2000 p95
default	36.120	3.444
12 FISTA steps	36.118	3.445
8 FISTA steps	36.047	3.461
overlap 12	35.987	3.501
overlap 8	35.999	3.486

Setting	PSNR (dB)	ΔE2000 p95
`--fast` (overlap 8, 12 steps)	**35.996**	**3.486**
overlap 8, 8 steps	35.912	3.503
overlap 4, 12 steps	35.930	3.504
48 × 48 tiles, overlap 8, 12 steps	35.940	3.534
4 ADMM rounds	35.794	3.552
2 ADMM rounds	34.378	4.045

`--fast` was chosen as the cheapest setting that keeps SSIM within 0.0002 of the default; the three settings below 0.9 s each lose a little more on every metric for 60–70 ms. It is a decode flag in automatic mode: it sets overlap 8 and 12 FISTA steps, and an explicit `--overlap` or `--fista-iters` overrides it.

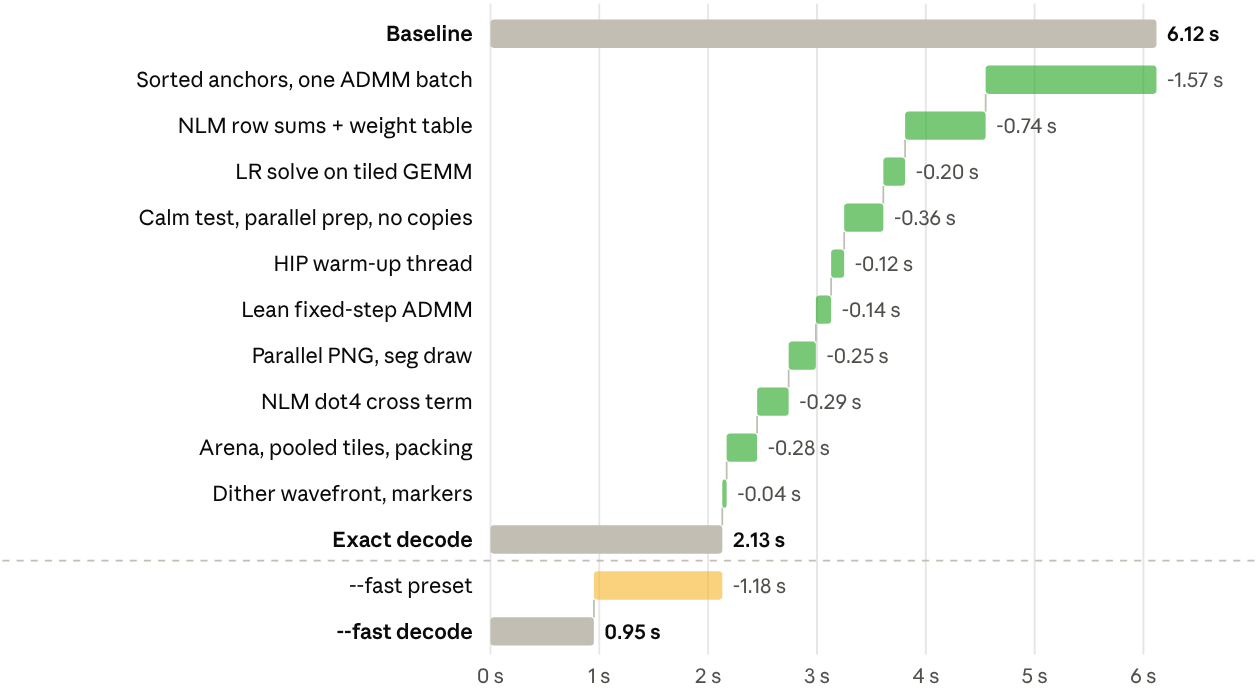
8. Results

The default decode is 2.87× faster with an identical output, and `--fast` is 6.4× faster than the original. Two changes, the anchor lookup and the NLM rewrite, account for 2.6 s of the 4.0 s removed by exact means.

Exact changes took the decode from 6.1 s to 2.13 s; --fast reaches 0.95 s

Wall time after each change, default settings, IMG_3690

green: exact reduction (same output bytes) · amber: changes the output · grey: totals



unprofiled wall time after each committed change; steps grouped in the order they were made

Where the remaining 0.95 s of a `--fast` decode goes, from stage marks on the final code:

Part	Time	Runs on
ADMM loop: DCT GEMMs	≈240 ms	GPU
ADMM loop: NLM prox	≈150 ms	GPU
ADMM loop: other kernels	≈30 ms	GPU
Unpack sample positions + tile prep	≈125 ms	CPU
LR stage (batch build, solve, merge)	≈80 ms	CPU + GPU
SR setup (prep, layout, AVIR, dither)	≈75 ms	CPU + GPU
Process start, PNG read, key derivation	≈60 ms	CPU
Composite + calm + PNG write	≈50 ms	GPU + CPU
Background frees, process exit	≈60 ms	CPU

GPU stages are still about half of the critical path, as at the start, but the remaining CPU stages are now mostly inherently serial: the Mersenne-Twister index draw (60 ms) must replay the encoder's random stream, and key derivation (31 ms) is a security parameter. Final-build timings were stable to about ±15 ms over 5 to 7 runs (default 2,096–2,186 ms, `--fast` 941–1,017 ms).

9. Discussion

9.1 The neural network is not in these timings

No neural network runs in any timing above. The decode uses `--photo-upscaler cs-sr --sr-pnp admm` without `--sr-prior-model`, so the ADMM proximal step is the classical NLM denoiser and the 2× warm start is AVIR, a linear resampler. The optional Swin2SR prior (ONNX on DirectML) takes a different path: the batched SR pass declines tiles that need it, and each tile goes through the earlier per-tile pipeline with a batching broker. Measured on the final build with the same container and anchors:

Decode	Prior	Wall time	PSNR	SSIM	ΔE p95
default	none (NLM)	2.13 s	36.120 dB	0.9285	3.444
default	Swin2SR fp32	18.9 s	35.833 dB	0.9284	3.439
`--fast`	none (NLM)	0.95 s	35.996 dB	0.9283	3.486
`--fast`	Swin2SR fp32	10.8 s	35.614 dB	0.9281	3.476

On this anchored container the network costs about 9× the time and does not improve quality (lower PSNR, equal SSIM and ΔE): the 300,000 exact anchors already pin the detail the prior would hallucinate. Optimizing the prior path was outside this work; the broker path, the folded DirectML graph and its batch size are documented separately in `docs/gpu-optimizations.md`.

9.2 Hybrid CPU/GPU execution

Splitting tiles between CPU and GPU does not pay here. The CPU implementation of the same ADMM loop needed about 600 thread-seconds for this workload, so the GPU is roughly two orders of magnitude faster per tile; giving the CPU even 1% of the tiles would lengthen the critical path. Hybrid execution paid off in a different form: CPU work that

does not depend on the GPU result overlaps it (HIP start-up and arena reservation during key derivation, chroma index draws alongside the luma draw, frees during the PNG write), and every remaining CPU stage uses all 24 threads.

9.3 Limitations

- One image, one container mode and one GPU. The relative gains depend on the anchor count (the anchor fix scales with it) and on tile geometry.
- The marker-event gain is measured, repeatable and unexplained; it may not transfer to other drivers or to Linux.
- The parallel PNG writer adds a dependency on AMD's AOCL-Compression to the GPU build.
- `--fast` changes the output; its cost was measured on this image only.

9.4 Future work

- *Even/odd DCT factorization.* Halves the GEMM arithmetic (about 240 ms of the 0.95 s), at the price of different rounding.
- *Keep tiles on the device.* LR tiles go to the host and back between stages, and SR tiles go to the host before the compositor uploads them again.
- *Pipelining by tile rows.* Compositing and PNG-encoding finished bands while the GPU solves later ones would hide most of the remaining CPU tail.
- *The neural prior path.* Port it to the batched pass, then re-evaluate whether it ever beats the anchored NLM decode.

10. Conclusion

A GPU decoder that looked GPU-bound was not: of its 6.1 s, the largest single cost was a quadratic host loop, and five of the ten largest costs were CPU work or driver overhead. Measured stage by stage and held to a byte-identical output, the decode dropped to 2.13 s (2.9×). The two decode knobs that barely affect quality, tile overlap and inner FISTA steps, take it to 0.95 s (6.4×) for 0.12 dB PSNR and 0.0002 SSIM, exposed as `--fast`. The remaining time is dominated by DCT GEMMs that run at the FMA limit of the current kernel design; going further either changes rounding or keeps data on the device across stages.

Appendix A. Reproducibility

Code: commit `bcc29df` on branch `local-sync/clang-gpu`. Build with `build_gpu.bat main` (needs ROCm 7.2, LLVM clang-cl, OpenCV 4.9 at `C:\opencv`, AOCL-Compression). Environment for the anchored decode:

```
CS_GPU_TILES=1  CS_GPU_ADMM=1  CS_HR_NO_POSTHOC=1
CS_HR_SIDECAR=<run folder>\003_IMG_3690_anchors_300k_sidecar.bin
```

Commands:

```
ImgReconstruct_backend.exe decrypt <container.png> out.png --password <pw> --
no-preview --tv 0 --photo-upscaler cs-sr --sr-pnp admm --hr-weight 1
ImgReconstruct_backend.exe decrypt <container.png> out.png --password <pw> --
no-preview --tv 0 --photo-upscaler cs-sr --sr-pnp admm --hr-weight 1 --fast
```

Verification: `cs_tests` (CPU build, `build_cpu.bat`) passes 611 of 611; GPU output compared byte for byte against the pre-optimization decode. Per-kernel ADMM timing: set `CS_GPU_PROF=1`. Engineering notes with the step-by-step table: `docs/gpu-optimizations.md`, Section 8.